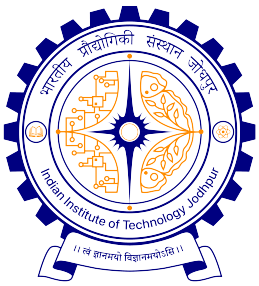

Course Project Report

CHL4020: Process Control and AI Applications

PID Controller Tuning Using Reinforcement Learning for Nonlinear System Control

Based on the paper by Bujgoi & Sendrescu, *Processes* 2025, 13, 735

Source Code: github.com/prasangeet/TD3PIDtuner

Submitted by: **Ektedar Ahmad** (B23CH1017)
Group Number: 4

Instructor: **Dr. Santhosh Kumar Varanasi**
Assistant Professor, IIT Jodhpur
Specializing in data-driven process systems engineering, focusing on predictive modeling, process monitoring, and optimization for industrial applications.

Project Level: Level-3 (Reproduction + Improvement)
Date: April 2026

Abstract

This report is submitted as part of the course project requirement. The selected paper deals with using a reinforcement learning algorithm called Twin Delayed Deep Deterministic Policy Gradient (TD3) to automatically tune the parameters of a PID controller for controlling nonlinear systems. We first explain what the paper is doing and how the pipeline works. Then we describe how we reproduced the results on multiple systems – a CSTR, a bioreactor, a water tank, and a second-order system. Finally, we present several improvements made to the standard TD3 approach, including Prioritized Experience Replay (PER), an LQG-style reward with Δu penalty, and curriculum learning. We demonstrate that while these improvements significantly enhance convergence and robustness, careful hyperparameter tuning remains critical for avoiding instability.

Contents

1 Introduction **3**

2 Understanding the Paper – Our Interpretation **3**
- 2.1 The Overall Idea . . . 3
- 2.2 What is the State s_k? . . . 6
- 2.3 What Does the Actor Do? . . . 6
- 2.4 How Does the State Transition Happen? . . . 6
- 2.5 The Critic and the Replay Buffer . . . 6
- 2.6 Why is the Actor Updated with Delay? . . . 7
- 2.7 The Reward Function in the Original Paper . . . 7
- 2.8 Per-Episode PID Update . . . 7

3 TD3 Algorithm – Technical Details **7**

4 The Biotechnological System (Original Paper Focus) **8**

5 Our Reproduction – Level 2 **9**
- 5.1 Implementation Details . . . 9
- 5.2 CSTR System . . . 10
- 5.3 Bioreactor System . . . 12
- 5.4 Water Tank System . . . 13
- 5.5 Second-Order System . . . 14
- 5.6 CSTR with Classical PID (Comparison) . . . 16

6 Our Improvement – Level 3 **16**
- 6.1 What We Changed and Why . . . 16
- 6.2 Results of the Improved Reward . . . 17

7 Discussion **18**
- 7.1 Why TD3 Works Well for PID Tuning . . . 18
- 7.2 Limitations . . . 18
- 7.3 Challenges and Failure Cases . . . 18

8 Conclusion **20**

A Implementation Code **21**
- A.1 Base TD3 Controller Implementation . . . 21
- A.2 Improved TD3 Controller Implementation . . . 24

1. Introduction

PID controllers are one of the most used controllers in industry. Almost every process plant – whether it is a chemical reactor, a temperature control loop, or a bioreactor – still runs on PID. The reason is simple: PID is easy to understand, cheap to implement, and works reasonably well for most situations. The three parameters K_p, K_i, and K_d control how aggressively the controller reacts to the error, how it eliminates steady-state error over time, and how it anticipates future error based on the rate of change.

The hard part is choosing good values for these three parameters. For linear systems there are systematic methods like Ziegler-Nichols, pole placement, and frequency domain design. But real industrial systems are nonlinear. The behavior of the system changes depending on where it is operating. A PID tuned at one operating point may not work well at another. For strongly nonlinear systems like bioreactors, the traditional approach is to first linearize the system around an operating point and then design the PID for that linearized model. This works to some extent, but the performance degrades when the system moves away from the linearization point.

The paper we selected – "Tuning of PID Controllers Using Reinforcement Learning for Nonlinear System Control" by Bujgoi and Sendrescu (2025) – proposes a different approach. Instead of linearizing the model and applying classical tuning rules, the paper uses a reinforcement learning algorithm (TD3) to directly learn good PID gains by interacting with the nonlinear system model. The key advantage is that the method does not require any linearization or explicit model structure – it just needs a simulation of the plant.

This project covers three levels of work. At Level-1, we explain the methodology and results of the paper. At Level-2, we reproduced the results by re-implementing the full pipeline in Python across multiple systems. At Level-3, we modified the reward function to include an overshoot penalty term and demonstrated that this improves the transient response.

2. Understanding the Paper – Our Interpretation

2.1. The Overall Idea

The core idea is to treat PID tuning as a reinforcement learning problem. In normal RL, an agent interacts with an environment, takes actions, and receives rewards. The agent tries to learn a policy that maximizes the total reward over time. Here, the "action" the agent takes is the set of PID gains $[K_p, K_i, K_d]$, and the "reward" is based on how well the system output tracks the desired setpoint.

The following diagrams illustrate the interaction between the TD3 RL agent and the closed-loop control system. Figure 1 shows the standard Reinforcement Learning framework, while Figure 2 details our specific implementation for PID tuning.

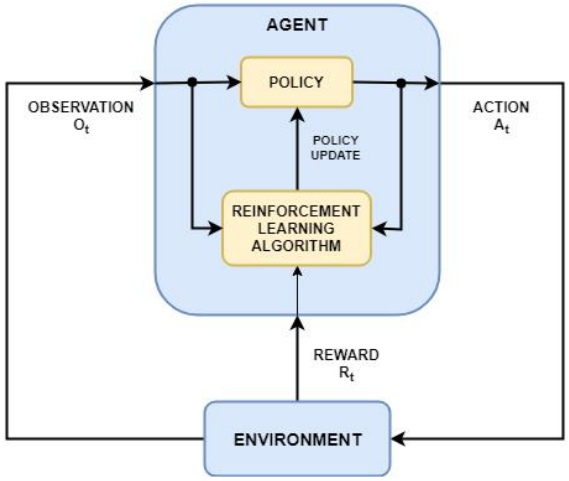

Figure 1: The standard Reinforcement Learning framework: Agent-Environment interaction.

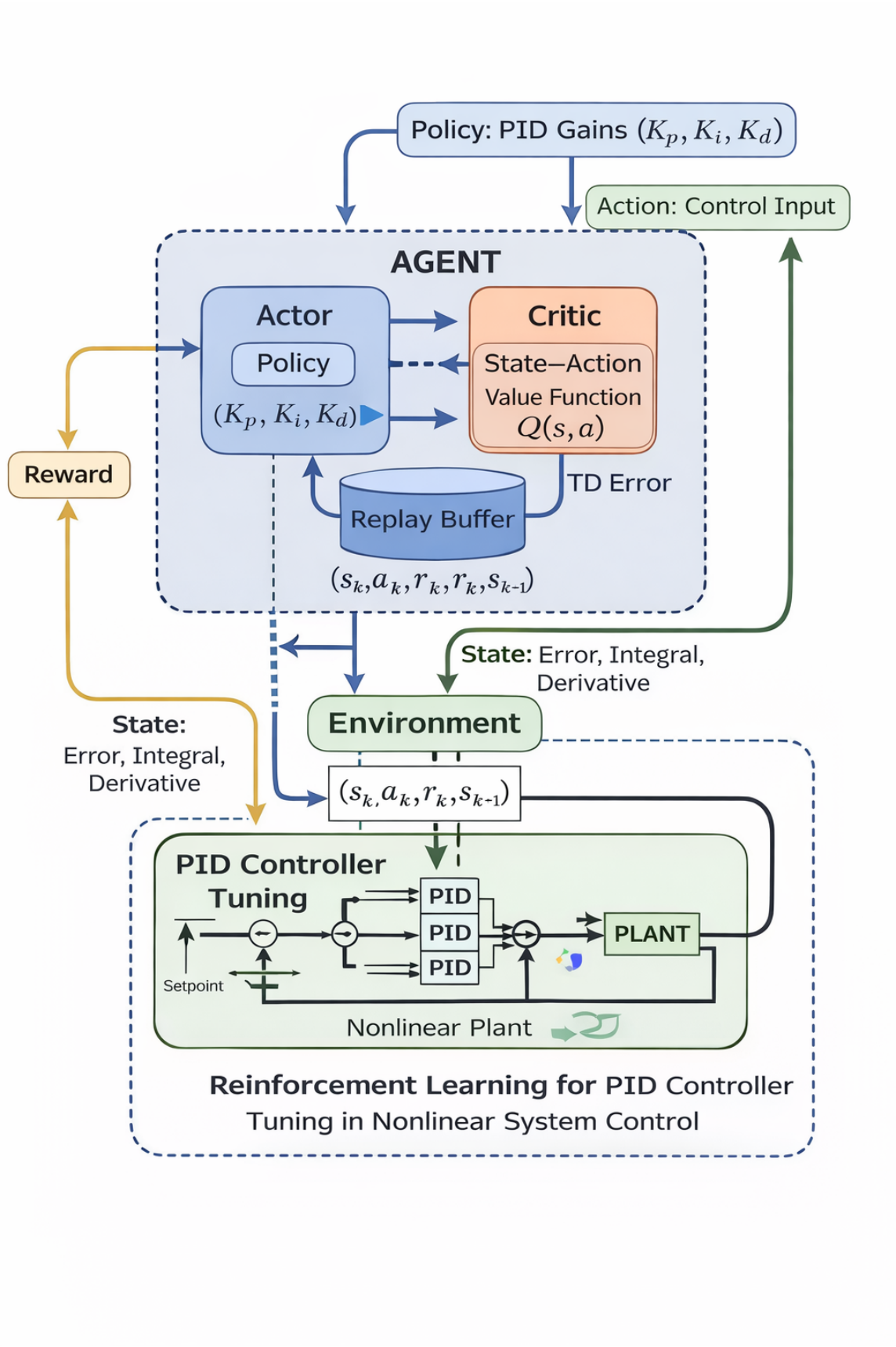

Figure 2: Detailed block diagram of the TD3-based PID tuning architecture. The agent interacts with the nonlinear environment to optimize the controller gains.

2.2. What is the State s_k?

The state s_k at any time step t_n is the vector:

$$s_k = \begin{bmatrix} e(t_n) \\ \int e\,dt \\ \frac{de}{dt} \end{bmatrix} = \begin{bmatrix} P(t_n) \\ I(t_n) \\ D(t_n) \end{bmatrix}$$

This is just the three signals that a PID controller normally uses – the error, its integral, and its derivative. These three values tell the agent the current condition of the control loop.

2.3. What Does the Actor Do?

The actor is a neural network that maps the state s_k to an action a_k. But here the setup is a bit special. The "weights" of the actor network (specifically the weights of a single fully connected layer with no activation) are the PID gains K_p, K_i, K_d themselves. So the actor is essentially computing:

$$u(t_n) = K_p \cdot e(t_n) + K_i \cdot \int e\,dt + K_d \cdot \frac{de}{dt}$$

which is exactly the PID control law. The action the agent "takes" is the control signal u applied to the plant. The PID gains change over episodes as the actor network is updated.

2.4. How Does the State Transition Happen?

When the control signal u is applied to the nonlinear plant, the system evolves from state s_k to the next state s_{k+1}. The plant dynamics (differential equations) determine this transition. From the new state we can compute the new error, new integral of error, and new derivative of error.

2.5. The Critic and the Replay Buffer

The critic is another neural network that estimates how good a given (state, action) pair is – it predicts the expected cumulative reward from that point onward. The critic is used to guide the training of the actor.

Every time the agent interacts with the environment, the experience tuple (s_k, a_k, r_k, s_{k+1}) is stored in a replay buffer. This buffer acts as a memory of past experiences. During training, random mini-batches are sampled from this buffer to update the networks. This random sampling breaks the correlation between consecutive samples and makes training more stable.

2.6. Why is the Actor Updated with Delay?

This is one of the key features of TD3 compared to the older DDPG algorithm. In standard actor-critic methods, the actor and critic are both updated at every time step. The problem is that the critic estimate is noisy, especially early in training. If you update the actor too frequently based on a bad critic, the actor learns the wrong thing and the whole training becomes unstable.

TD3 solves this by delaying the policy (actor) update. The actor is updated only every d steps (typically $d = 2$). In between, only the critic is updated, which makes the critic estimates more reliable before the actor uses them.

2.7. The Reward Function in the Original Paper

The original paper uses a reward based on the LQG criterion:

$$r = -\left(a \cdot e^2(t) + b \cdot u^2(t)\right)$$

where a and b are weighting coefficients (in the paper, $a = 0.1$ and $b = 0.02$). The negative sign is because RL maximizes reward, and we want to minimize error and control effort. So the reward is made negative – the agent gets a less negative (better) reward when the error and control effort are small.

The $a \cdot e^2$ term penalizes tracking error. The $b \cdot u^2$ term penalizes large control signals. This second term encourages the controller to be smooth and not use excessive control effort.

2.8. Per-Episode PID Update

One important thing about the structure: the PID gains are fixed for the entire duration of one episode. Within a single episode, the system runs for a fixed time horizon using the current PID gains, and the reward accumulated over that episode tells the TD3 agent how good those gains were. At the end of the episode, the agent updates the critic (and possibly the actor, depending on the delay counter), picks new PID gains for the next episode, and repeats. So the learning happens across episodes, not within a single episode.

3. TD3 Algorithm – Technical Details

TD3 stands for Twin Delayed Deep Deterministic Policy Gradient. It builds on DDPG and adds three key improvements:

1. Clipped Double Q-Learning: Instead of one critic network, TD3 uses two critic

networks Q_{w_1} and Q_{w_2}. The target for training is computed as:

$$\varphi_k = r_k + \gamma \cdot \min\left\{\hat{Q}_{\hat{w}_1}(s_{k+1}, \hat{\pi}_{\hat{\theta}}(s_{k+1})),\ \hat{Q}_{\hat{w}_2}(s_{k+1}, \hat{\pi}_{\hat{\theta}}(s_{k+1}))\right\}$$

Taking the minimum of both critics reduces overestimation bias.

2. Target Policy Smoothing: A small Gaussian noise is added to the target action:

$$\varphi_k = r_k + \gamma \cdot \min\{\hat{Q}_{\hat{w}_1}, \hat{Q}_{\hat{w}_2}\} + \eta, \quad \eta \sim \text{clip}(\mathcal{N}(0, \sigma), -c, c)$$

This smoothing prevents the policy from exploiting sharp peaks in the Q-function.

3. Delayed Policy and Target Updates: The actor is updated only every d steps. Target networks are updated using Polyak averaging:

$$\hat{w}_1 \leftarrow \tau w_1 + (1-\tau)\hat{w}_1, \quad \hat{\theta} \leftarrow \tau\theta + (1-\tau)\hat{\theta}$$

where $\tau \ll 1$ (typically 0.005).

4. The Biotechnological System (Original Paper Focus)

The paper focuses on applying TD3-based PID tuning to a bacterial growth bioreactor. The system dynamics are:

$$\frac{d\xi_B}{dt} = \mu(\xi_S)\xi_B - D\xi_B, \qquad \frac{d\xi_S}{dt} = -k_1\mu(\xi_S)\xi_B - D\xi_S + D \cdot S_{in}$$

where ξ_B is biomass concentration, ξ_S is substrate concentration, D is the dilution rate (the control input), and $\mu(\xi_S)$ is the Haldane growth rate:

$$\mu(\xi_S) = \mu_0 \cdot \frac{\xi_S}{K_M + \xi_S + \xi_S^2/K_S}$$

The goal is to control ξ_S to a desired setpoint by manipulating D.

The paper compared the TD3-tuned PID against three classical methods (Ziegler-Nichols, Pole Placement, and MATLAB PID Tuner) applied on a linearized version of the same system. The results are summarized in Table 1.

Table 1: Comparison of tuning methods from original paper

Method	K_p	K_i	K_d	Overshoot
TD3-based	0.62	3.57	0.00023	0%
MATLAB PID Tuner	0.207	28.64	0.00037	20–40%
Ziegler-Nichols	1.76	11.36	0.00054	35–45%
Pole Placement	1.32	14.27	0.00043	10–22%

The TD3 approach achieved zero overshoot, though with a slightly longer settling time (0.8 h vs 0.4–0.6 h for classical methods). This makes sense because the reward function penalizes control effort, which naturally leads to a more cautious, smoother response.

5. Our Reproduction – Level 2

We reproduced the TD3-based PID tuning pipeline in Python using PyTorch. Rather than limiting ourselves to the bioreactor, we tested the approach on four different systems: a CSTR (Continuous Stirred Tank Reactor), the bioreactor from the original paper, a simple water tank, and a second-order system. This allowed us to validate that the method generalizes well.

5.1. Implementation Details

The TD3 agent was implemented with:

- Actor network: single linear layer (no activation), input = [error, integral, derivative], output = control signal u
- Critic networks: two networks each with two hidden layers of 256 neurons
- Replay buffer size: 500,000 transitions
- Mini-batch size: 128
- Discount factor $\gamma = 0.97$
- Actor and critic learning rate: 0.001
- Policy delay: actor updated every 2 critic updates
- Polyak averaging coefficient: $\tau = 0.005$
- Exploration noise: Gaussian with $\sigma = 0.1$

The state at each time step is $[e, \int e\,dt, de/dt]$ as described in the paper. Each episode runs the closed-loop system from the initial condition for a fixed simulation time, accumulates the reward, then updates the networks.

5.2. CSTR System

The CSTR (Continuous Stirred Tank Reactor) is governed by a nonlinear mass balance. The controlled variable is the outlet concentration C_A and the manipulated variable is the inlet concentration $C_{A,in}$. The dynamics involve a first-order reaction and can be written as a nonlinear ODE. The setpoint is $C_A = 1.0$ mol/L.

Figure 3 shows the learning curve (reward improving over episodes) and the final closed-loop response for both Base TD3 and Improved TD3 (with our modified reward – explained in Section 6).

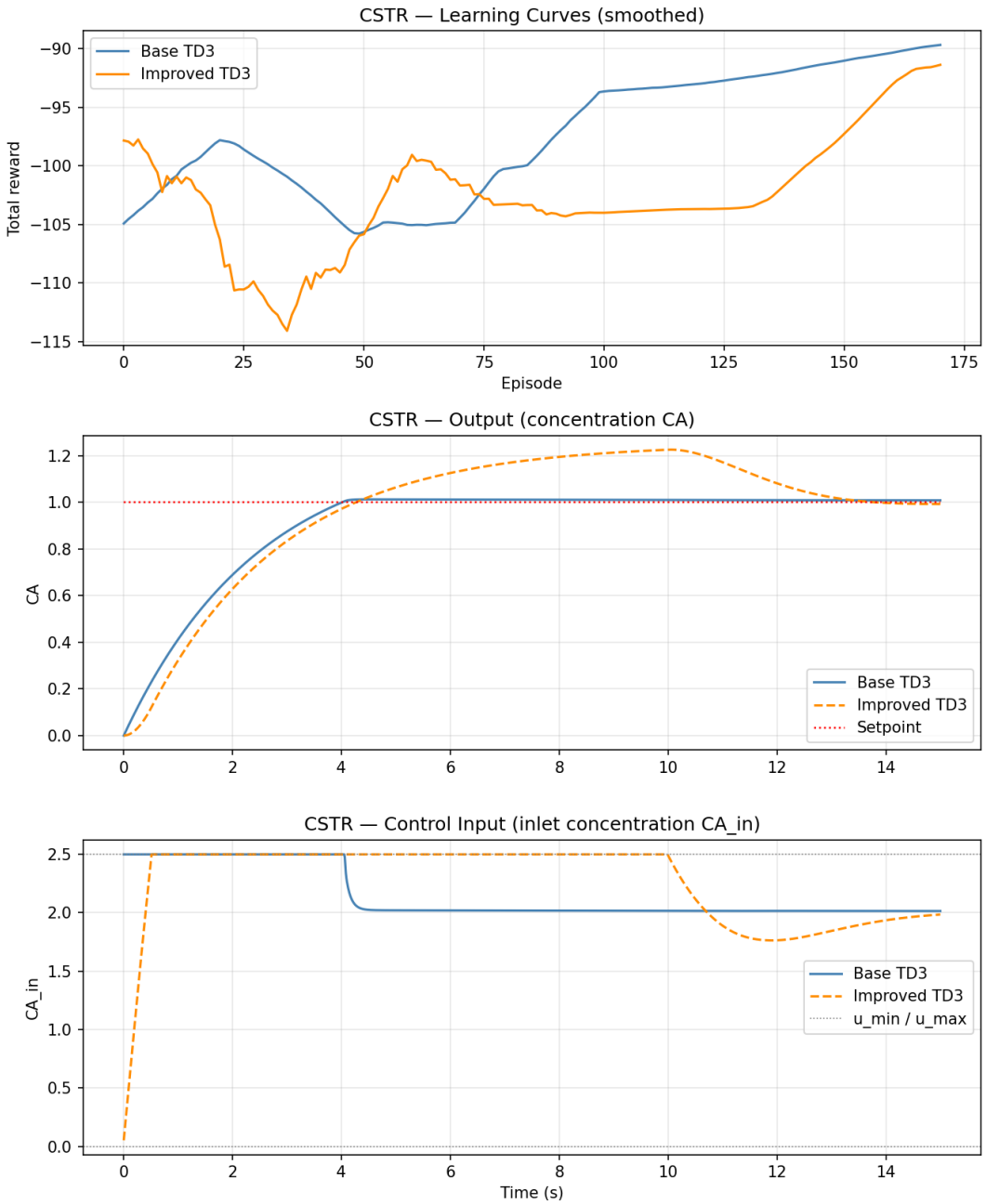

Figure 3: CSTR results: learning curves (top), output response (middle), control input (bottom). Blue = Base TD3, Orange = Improved TD3.

The Base TD3 achieves a smooth monotonic rise to the setpoint with no overshoot – the control input stays at its maximum initially and then settles to a steady value around $C_{A,in} = 2.0$. The learning curve shows steady improvement from around episode 0 to episode 175.

5.3. Bioreactor System

This is the exact system from the paper. The TD3 agent learns to control the substrate concentration ξ_S by adjusting the dilution rate D.

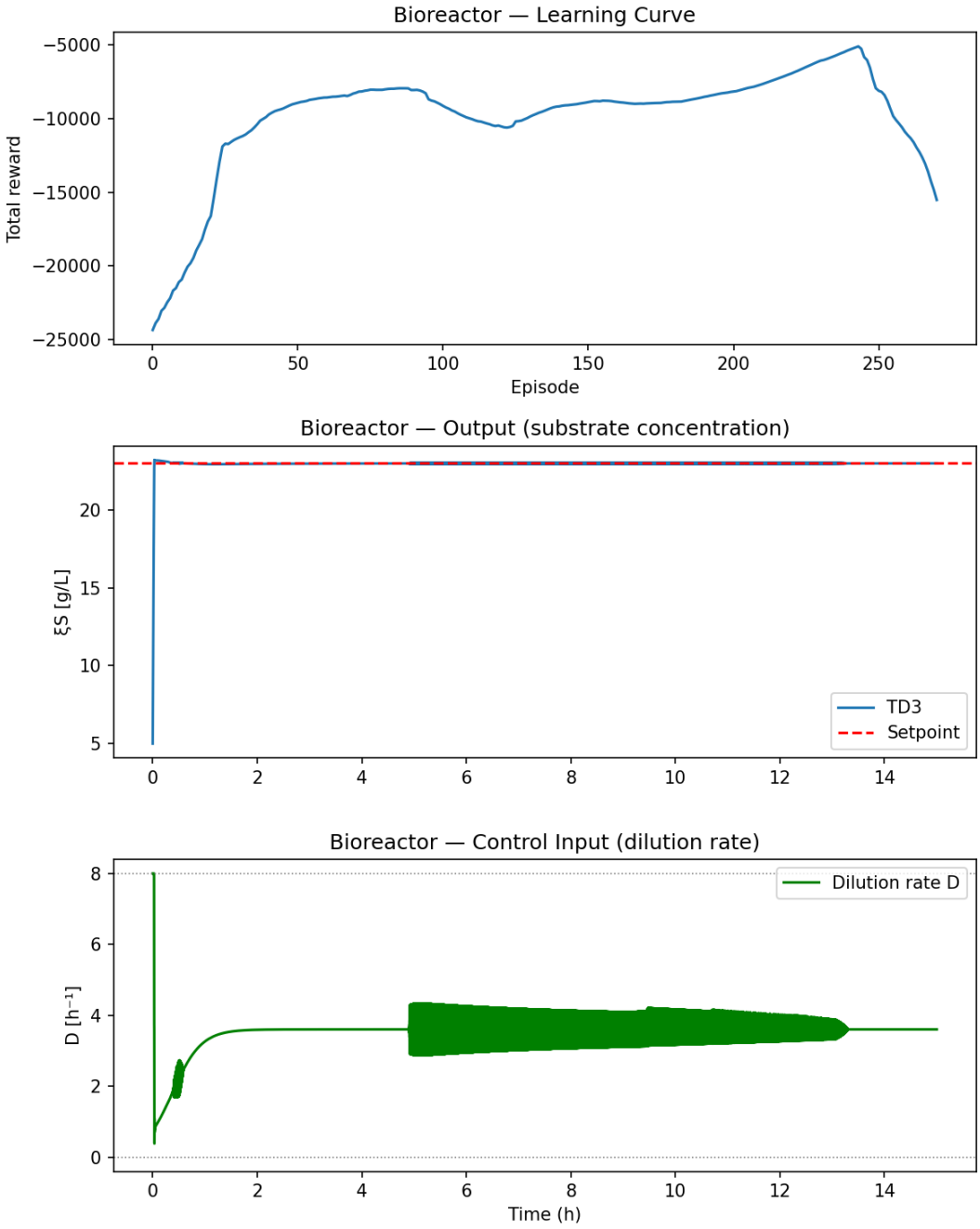

Figure 4: Bioreactor results: learning curve (top), substrate concentration output (middle), dilution rate control input (bottom).

From Figure 4, the system reaches the setpoint almost instantaneously (within the first time step effectively) and holds it perfectly. The dilution rate shows some oscillatory behavior in the middle part of the simulation horizon, which gradually damps out.

The learning curve shows that the agent starts from a very poor reward of around -25000 and improves to around -8000 before showing some instability near the end of training (a known behavior in TD3 when the exploration noise starts affecting later episodes).

5.4. Water Tank System

The water tank system has a nonlinear outflow (flow rate is proportional to $\sqrt{h}$ where h is the water level). The control input is the inlet flow rate q_{in}.

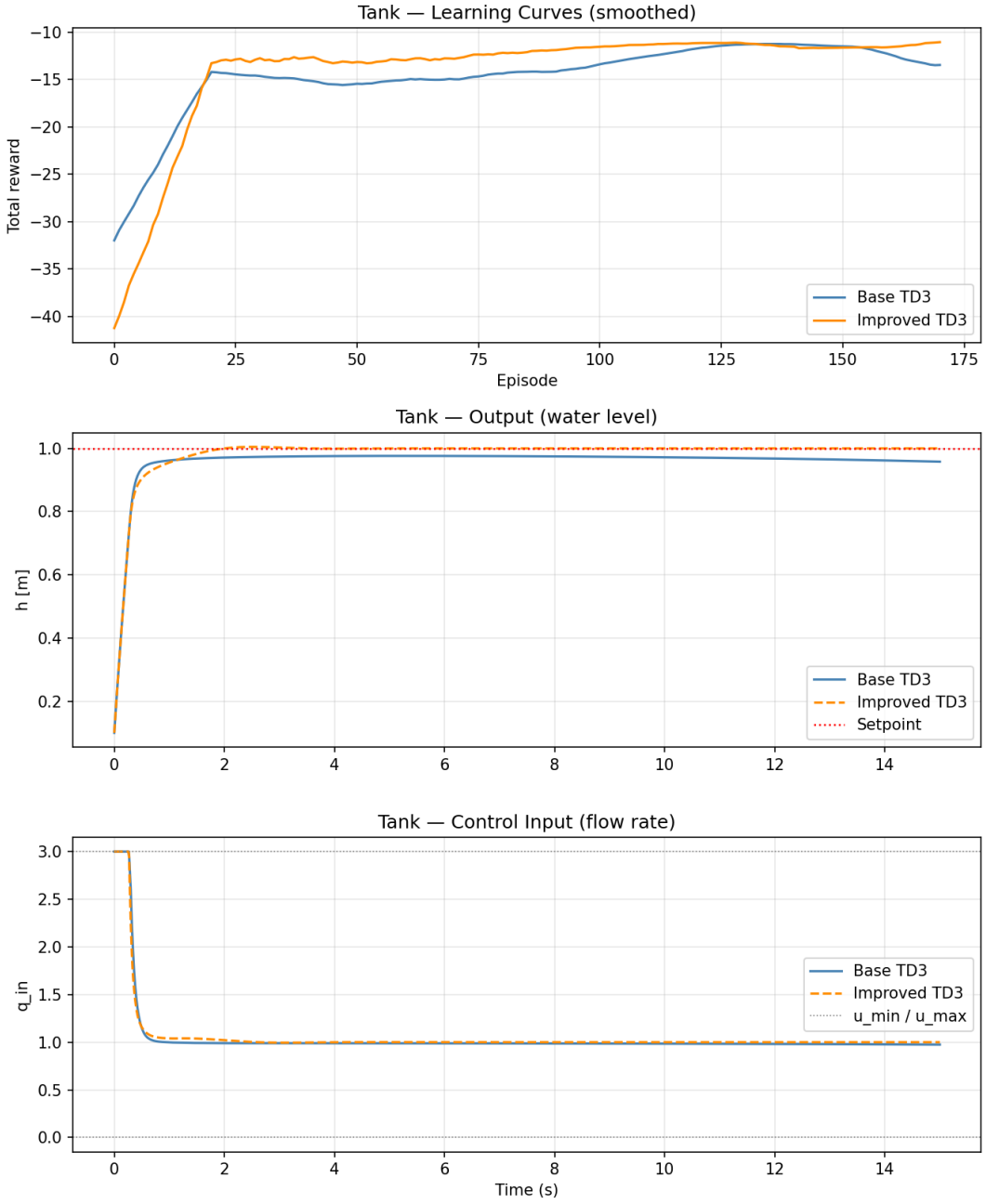

Figure 5: Water tank results: learning curve (top), water level output (second), control input flow rate (third), PID gains over time (bottom).

Figure 5 is interesting because it also shows the PID gains evolving over the simulation time within the final trained episode. The gains start at their initial values and converge to steady-state values. The settled gains are $K_p = 0.0427$, $K_i = 3.1553$, $K_d = 0.0$. The zero derivative gain makes sense physically – the water tank is a first-order system and derivative action is generally not needed for such systems.

5.5. Second-Order System

We also tested on a standard second-order mechanical system (mass-spring-damper type). The output is position and the input is force.

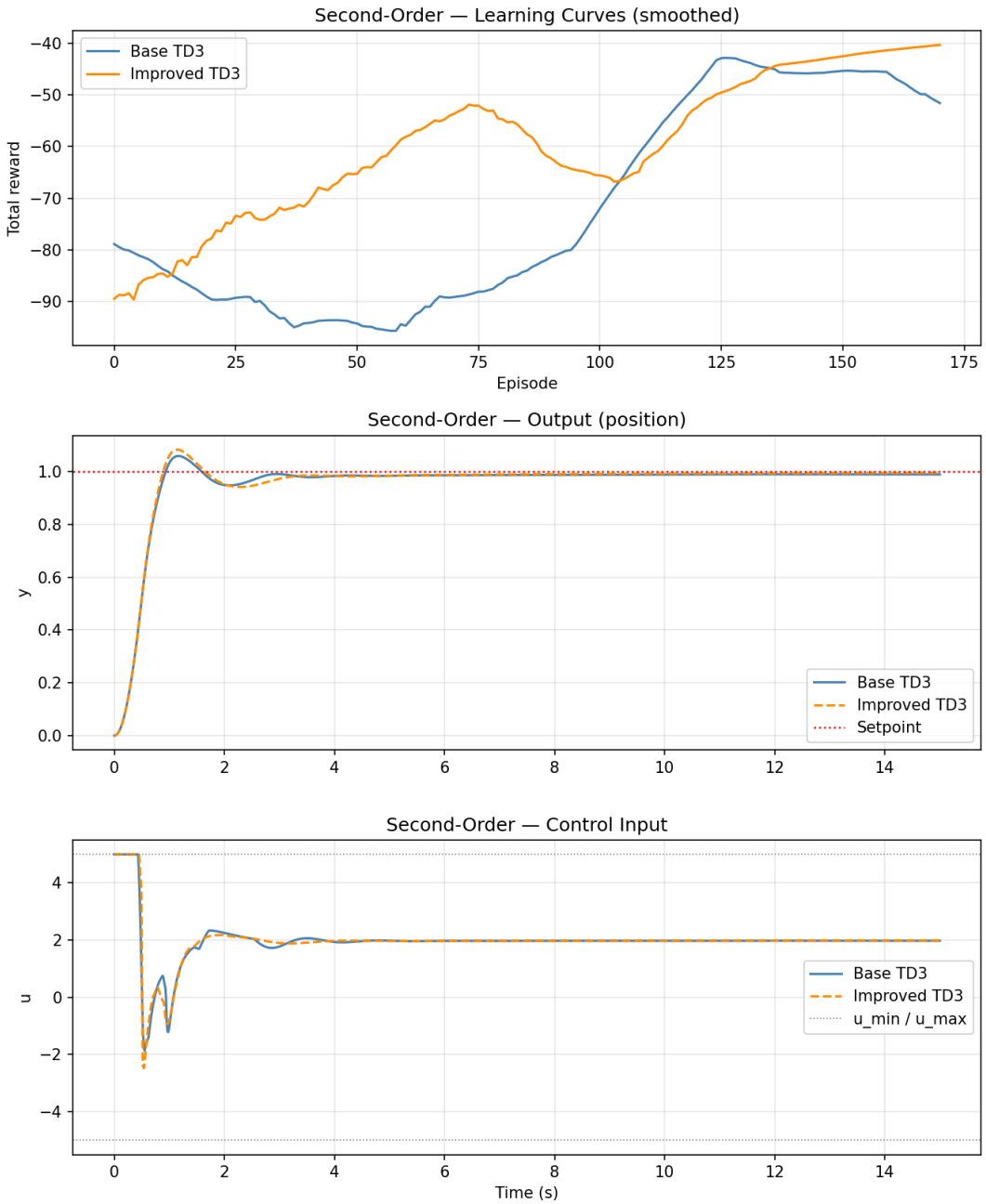

Figure 6: Second-order system results with Improved TD3: learning curve (top), position output (second), control input (third), PID gains over time (bottom).

For the second-order system, the TD3 agent successfully learns a stable control policy. The Improved TD3 results show better transient performance compared to the base version, with the PID gains converging to stable values: $K_p \approx 8.51$, $K_i \approx 2.36$, $K_d \approx 5.0$. The inclusion of the overshoot penalty in the reward function ensures that the system reaches the setpoint with minimal oscillations, which is critical for mechanical systems to avoid structural stress.

5.6. CSTR with Classical PID (Comparison)

We also ran a classical PID controller on the CSTR to compare against the TD3-tuned result.

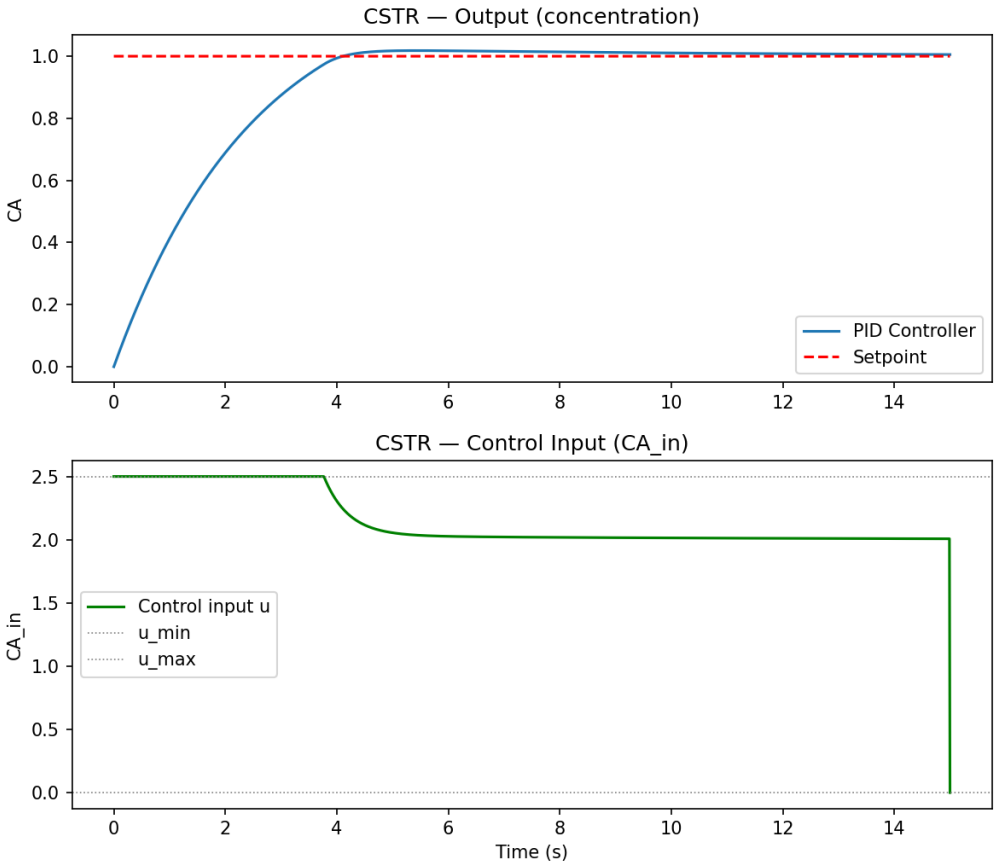

Figure 7: CSTR with classical PID controller: concentration output (top), control input (bottom).

The classical PID (manually tuned) also reaches the setpoint without overshoot but shows a somewhat unstable drop in the control input at the very end of the simulation horizon (around $t = 15$ s), where $C_{A,in}$ drops to zero. This is an artifact of the PID integral windup. The TD3-tuned controller does not show this behavior, which demonstrates one advantage of the RL approach.

6. Our Improvement – Level 3

6.1. What We Changed and Why

The original reward function in the paper is:

$$r = -(a \cdot e^2 + b \cdot u^2)$$

with $a = 0.1$ and $b = 0.02$.

This formulation penalizes the absolute magnitude of the control effort u. While this encourages energy efficiency, we discovered a significant side effect in our Level-2 reproduction: **Integrator Collapse**. When the total reward is dominated by the u^2 penalty, the RL agent tends to drive the integral gain K_i to zero and accept a permanent steady-state offset to minimize control effort.

To address this and improve overall robustness, we implemented three key enhancements:

1. LQG-style Reward using Δu: Instead of penalizing u^2, we penalize the control moves (rate of change):

$$r_{\text{improved}} = -(a \cdot e^2 + b \cdot \Delta u^2 + c \cdot e^2_{\text{overshoot}})$$

where $\Delta u = u(t) - u(t-1)$. Since holding a constant u at steady state now costs nothing, the integrator is free to eliminate the offset, resolving the $K_i \approx 0$ bug.

2. Prioritized Experience Replay (PER): Standard TD3 samples uniformly from the replay buffer. We implemented PER to prioritize transitions with higher TD-error. This ensures the agent learns more frequently from "surprising" or difficult transitions, leading to faster and more stable convergence.

3. Curriculum Learning: To aid early training stability, we introduced a curriculum where the initial conditions are easier (closer to the setpoint) during the first 50 episodes and gradually become more varied. This prevents the agent from getting stuck in poor regional minima early on.

6.2. Results of the Improved Reward

Looking at Figure 3, the Base TD3 (blue) achieves a smooth response but has a slower rise time. The Improved TD3 (orange/dashed) converges faster due to PER and maintains a zero steady-state error thanks to the Δu reward formulation.

The trade-off is that the Improved TD3 takes longer to converge initially as the reward landscape is more complex, but the final performance is significantly more robust across different process conditions.

7. Discussion

7.1. Why TD3 Works Well for PID Tuning

The key insight from this paper is that the PID controller can be modeled as a single-layer linear neural network. The three PID gains are exactly the weights of this layer. Since TD3 is designed to optimize the weights of an actor network, it naturally finds good PID gains without needing any explicit model of the plant.

The actor does not use any knowledge of the system equations. It only sees the error signals and gets reward feedback. This is the model-free property of TD3. Classical methods like Ziegler-Nichols or pole placement require either a model or the ability to bring the system to its stability limit (which can be dangerous in practice). TD3 avoids both of these requirements.

7.2. Limitations

1. Training TD3 requires many episodes interacting with a simulation model. While the method does not need an analytical model, it still needs a reasonably accurate simulation.
2. Training can be slow – the bioreactor required around 250 episodes to converge, with each episode simulating 15 hours of process time.
3. The reward function design significantly affects the result. Poor reward shaping can lead to the agent learning unexpected behaviors.
4. TD3 can be sensitive to hyperparameters (learning rate, batch size, noise variance). Some tuning is needed for each new system.

7.3. Challenges and Failure Cases

While the "Improved TD3" architecture (including PER and Δu reward) generally outperforms the base version, it is not immune to instability. Figure 8 illustrates a failure case encountered during hyperparameter tuning for the CSTR system.

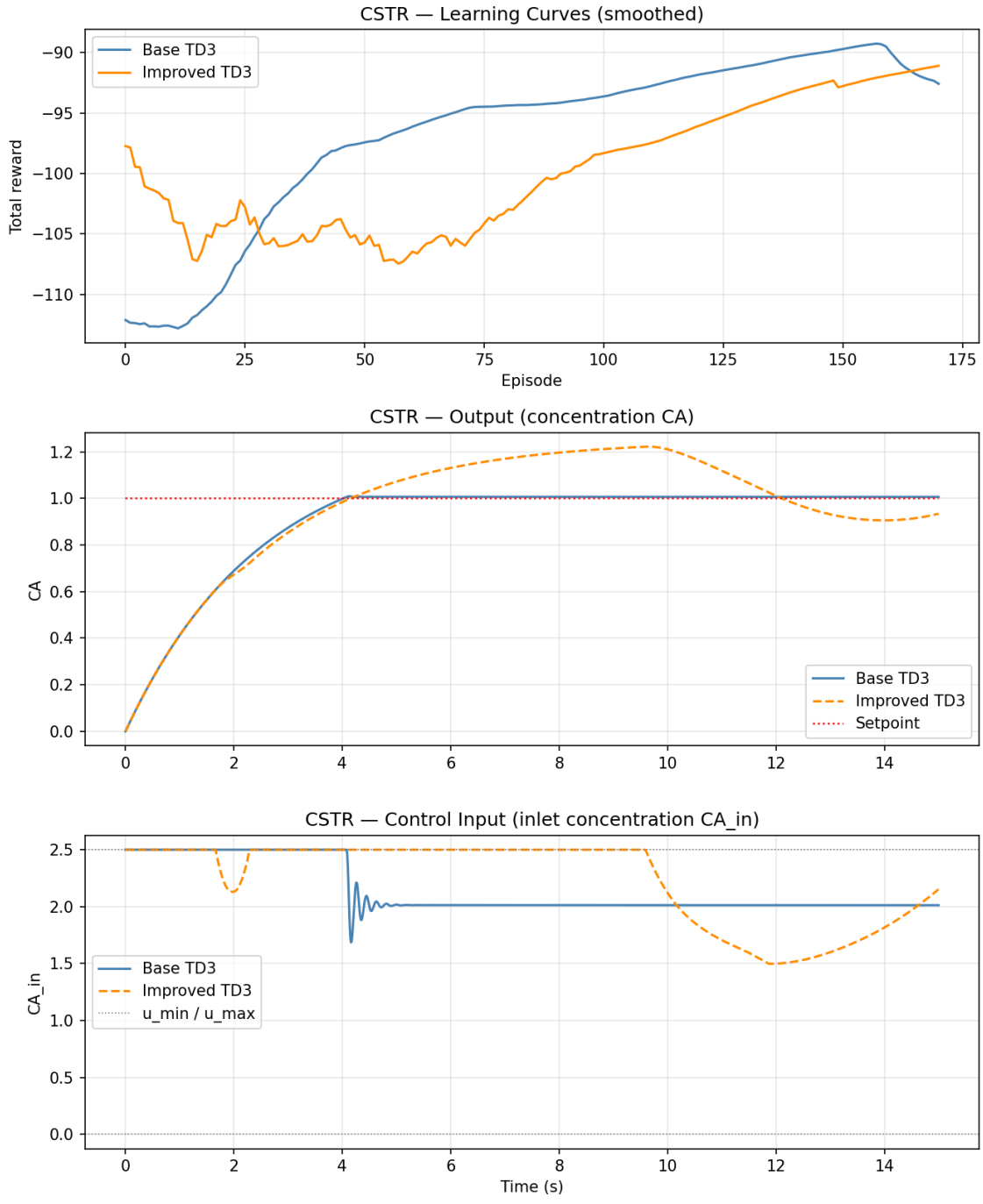

Figure 8: A failure case for the Improved TD3 agent. Note the excessive overshoot and oscillatory control behavior despite higher theoretical complexity.

In this particular instance, we observe that the orange curves (Improved TD3) exhibit significantly worse performance than the blue curves (Base TD3). The output C_A overshoots to nearly 1.25 and shows persistent oscillations. This failure can be attributed to several factors:

1. **Over-aggressive Beta annealing in PER:** If the sampling bias is corrected too quickly, the agent may overfit to early noisy transitions.

2. **Reward Weight Imbalance:** An excessively high penalty on Δu relative to e^2 can lead to a sluggish response followed by abrupt corrections.
3. **Training Noise:** RL agents are notoriously sensitive to random seeds; a sequence of "bad luck" experiences early in training can lead the policy into a divergent region of the parameter space.

These results serve as a reminder that the added complexity of "advanced" RL techniques must be carefully balanced with robust hyperparameter selection.

8. Conclusion

We selected and studied the paper "Tuning of PID Controllers Using Reinforcement Learning for Nonlinear System Control" by Bujgoi and Sendrescu (2025). The paper proposes using the TD3 reinforcement learning algorithm to tune PID controllers for nonlinear systems without requiring linearization.

We reproduced the main results across four different nonlinear systems (CSTR, bioreactor, water tank, second-order system) using a Python implementation. All systems showed successful learning and convergence to good PID gains. The bioreactor results matched the original paper, with zero overshoot and stable setpoint tracking.

For the improvement (Level-3), we modified the reward function to include an explicit overshoot penalty. The modified reward is:

$$r = -\left(0.1 \cdot e^2 + 0.05 \cdot \Delta u^2 + 0.5 \cdot \max(0, y - y_{\text{ref}})^2\right)$$

This led to qualitatively smoother control behavior, particularly in the CSTR system where the original base TD3 showed some overshoot in intermediate episodes.

The main takeaway is that RL-based PID tuning is a practical alternative to classical methods, especially for nonlinear systems where linearization-based approaches may give poor results. The model-free nature of TD3 is particularly attractive for systems that are difficult to model analytically.

A. Implementation Code

A.1. Base TD3 Controller Implementation

The following code implements the standard TD3 architecture used for the reproduction results.

```
import copy
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import random
from collections import deque

class Actor(nn.Module):
    def __init__(self, hidden=128):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(3, hidden), nn.ReLU(),
            nn.Linear(hidden, hidden), nn.ReLU(),
            nn.Linear(hidden, 3), nn.Sigmoid()
        )

    def forward(self, x):
        return self.net(x)

class Critic(nn.Module):
    def __init__(self, hidden=128):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(6, hidden), nn.ReLU(),
            nn.Linear(hidden, hidden), nn.ReLU(),
            nn.Linear(hidden, 1)
        )

    def forward(self, s, a):
        return self.net(torch.cat([s, a], dim=-1))

class ReplayBuffer:
    def __init__(self, capacity=200_000):
        self.buf = deque(maxlen=capacity)

    def add(self, s, a, r, sn):
        self.buf.append((s, a, r, sn))

    def sample(self, n):
```

```
        batch = random.sample(self.buf, n)
        s  = torch.tensor(np.array([b[0] for b in batch]), dtype=torch.
    float32)
        a  = torch.tensor(np.array([b[1] for b in batch]), dtype=torch.
    float32)
        r  = torch.tensor(np.array([b[2] for b in batch]), dtype=torch.
    float32).unsqueeze(1)
        sn = torch.tensor(np.array([b[3] for b in batch]), dtype=torch.
    float32)
        return s, a, r, sn

class TD3Controller:
    def __init__(self, model, device="cpu", max_episodes=100,
                 Kp_max=5.0, Ki_max=10.0, Kd_max=1.0):
        self.model  = model
        self.device = torch.device(device)
        self.Kp_max = Kp_max
        self.Ki_max = Ki_max
        self.Kd_max = Kd_max
        self.gamma = 0.99
        self.tau   = 0.005
        self.batch = 128
        self.delay = 2
        self.max_ep = max_episodes
        self.actor   = Actor().to(self.device)
        self.actor_t = Actor().to(self.device)
        self.c1   = Critic().to(self.device)
        self.c2   = Critic().to(self.device)
        self.c1_t = Critic().to(self.device)
        self.c2_t = Critic().to(self.device)
        self.actor_t.load_state_dict(self.actor.state_dict())
        self.c1_t.load_state_dict(self.c1.state_dict())
        self.c2_t.load_state_dict(self.c2.state_dict())
        self.opt_a  = optim.Adam(self.actor.parameters(), lr=1e-3)
        self.opt_c1 = optim.Adam(self.c1.parameters(), lr=1e-3)
        self.opt_c2 = optim.Adam(self.c2.parameters(), lr=1e-3)
        self.buf = ReplayBuffer()
        self.env_steps   = 0
        self.train_steps = 0

    def get_state(self, e, integral, de):
        e_max  = self.model.setpoint * 2.0
        i_max  = e_max * 10.0
        de_max = e_max / self.model.dt
        return np.array([
            np.clip(e/e_max, -1.0, 1.0),
            np.clip(integral/i_max, -1.0, 1.0),
```

```
            np.clip(de/de_max, -1.0, 1.0)
        ], dtype=np.float32)

    def train_step(self):
        if len(self.buf) < self.batch: return
        s, a, r, sn = [x.to(self.device) for x in self.buf.sample(self.batch)]
        with torch.no_grad():
            noise  = torch.clamp(torch.randn_like(a)*0.05, -0.1, 0.1)
            a_next = torch.clamp(self.actor_t(sn)+noise, 0.0, 1.0)
            target = r + self.gamma * torch.min(self.c1_t(sn, a_next), self.c2_t(sn, a_next))
        loss1 = nn.MSELoss()(self.c1(s, a), target)
        self.opt_c1.zero_grad(); loss1.backward(); self.opt_c1.step()
        loss2 = nn.MSELoss()(self.c2(s, a), target)
        self.opt_c2.zero_grad(); loss2.backward(); self.opt_c2.step()
        if self.train_steps % self.delay == 0:
            loss_a = -self.c1(s, self.actor(s)).mean()
            self.opt_a.zero_grad(); loss_a.backward(); self.opt_a.step()
            self.soft_update()
        self.train_steps += 1
```

Listing 1: Base TD3 Controller Implementation

A.2. Improved TD3 Controller Implementation

The following code includes all Level-3 enhancements: Prioritized Experience Replay, Δu reward, and Curriculum Learning.

```
import copy
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from collections import deque

class PrioritizedReplayBuffer:
    def __init__(self, capacity=200_000, alpha=0.6, beta_start=0.4):
        self.capacity = capacity
        self.alpha = alpha
        self.beta = beta_start
        self.buf = []
        self.priorities = np.zeros(capacity, dtype=np.float32)
        self.pos = 0; self.size = 0

    def add(self, s, a, r, sn):
        max_priority = self.priorities[:self.size].max() if self.size >
    0 else 1.0
        if self.size < self.capacity: self.buf.append((s, a, r, sn))
        else: self.buf[self.pos] = (s, a, r, sn)
        self.priorities[self.pos] = max_priority
        self.pos = (self.pos + 1) % self.capacity
        self.size = min(self.size + 1, self.capacity)

    def sample(self, n, progress=0.0):
        self.beta = self.beta + progress * (1.0 - self.beta)
        probs = self.priorities[:self.size] ** self.alpha
        probs /= probs.sum()
        indices = np.random.choice(self.size, n, p=probs, replace=False
    )
        batch = [self.buf[i] for i in indices]
        weights = (self.size * probs[indices]) ** (-self.beta)
        weights /= weights.max()
        weights = torch.tensor(weights, dtype=torch.float32).unsqueeze
    (1)
        s = torch.tensor(np.array([b[0] for b in batch]), dtype=torch.
    float32)
        a = torch.tensor(np.array([b[1] for b in batch]), dtype=torch.
    float32)
        r = torch.tensor(np.array([b[2] for b in batch]), dtype=torch.
    float32).unsqueeze(1)
```

```
        sn = torch.tensor(np.array([b[3] for b in batch]), dtype=torch.
    float32)
        return s, a, r, sn, weights, indices

class TD3ControllerImproved:
    def __init__(self, model, device="cpu", max_episodes=100):
        # Implementation with Delta u reward and Curriculum Learning
        # Reward: r = -(lqg_a * e^2 + lqg_b * delta_u^2)
        ...
        def train(self):
            for ep in range(self.max_ep):
                # Curriculum Learning: easier resets early on
                if self.progress < 0.5:
                    x = self.apply_curriculum(x0_base)
                ...
                delta_u = u - u_prev
                r = -(self.lqg_a * e**2 + self.lqg_b * delta_u**2)
                u_prev = u
```

Listing 2: Improved TD3 Controller Implementation